SIMATS ENGINEERING

(SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 Assessment Tool 1 — Git & Docker Technical Assignment

Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System

Technical Report — Version Control & Containerization

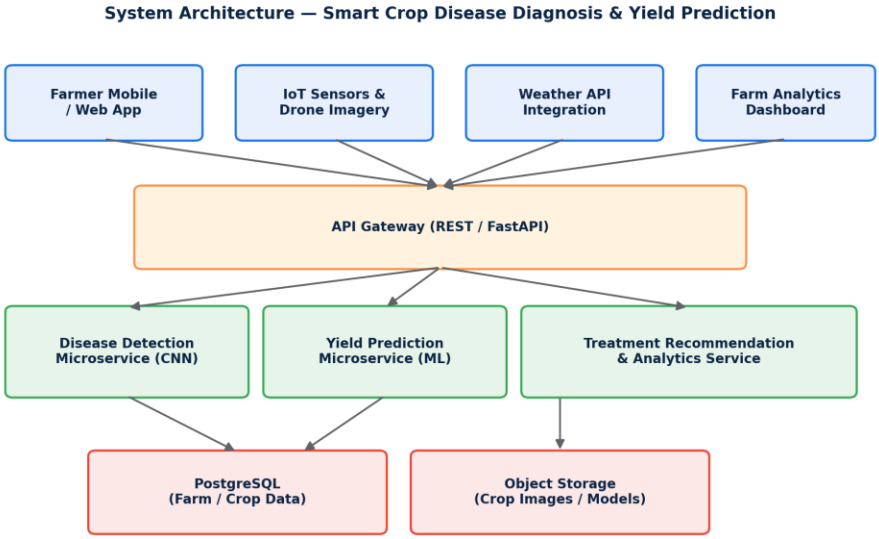

Figure 1: High-level system architecture of the proposed platform

1. Problem Analysis

1.1 Industry Context

Agriculture is increasingly adopting AI technologies to improve crop productivity and minimize losses caused by diseases and pests. Computer vision, drones, IoT sensors, and weather analytics help farmers monitor crop conditions and implement precision farming practices. As the volume of field data grows, software platforms that can reliably version, test, package, and deploy AI models become essential to translating research prototypes into dependable farm-facing tools.

1.2 Problem Statement

Crop diseases significantly reduce agricultural productivity and often spread before farmers can detect them through manual inspection. Existing methods are time-consuming and require expert intervention. The proposed AI-based crop management platform analyzes crop images, monitors environmental conditions, predicts disease outbreaks, estimates crop yield, and recommends suitable treatments. The system is designed to support sustainable agriculture while improving farm productivity and profitability.

1.3 Project Objectives

- Detect crop diseases using AI-based image classification.
- Predict crop yield accurately using historical and sensor data.
- Monitor environmental conditions in real time via IoT sensors.
- Recommend suitable disease treatments to farmers.
- Analyze the impact of weather patterns on crop health.
- Generate farm analytics reports for decision support.
- Reduce crop losses through early intervention.
- Improve overall agricultural productivity and profitability.

1.4 Functional Requirements

- Image upload and AI-based disease classification module.
- Yield prediction engine using regression / ML models.
- Real-time IoT sensor data ingestion (soil moisture, temperature, humidity).
- Weather API integration for forecast-based risk analysis.
- Treatment recommendation engine mapped to detected disease classes.
- Dashboard for farm analytics, reports, and historical trends.
- Role-based access for farmers, agronomists, and administrators.

1.5 Expected Outcomes

- Early identification of crop diseases before large-scale spread.
- Increased crop yield through timely, data-driven interventions.
- Reduced pesticide usage via targeted treatment recommendations.
- Better farming decisions supported by analytics dashboards.
- Improved farm profitability and long-term sustainability.
- Enhanced food security and efficient crop management at scale.

2. Project Structure Design

2.1 Repository Folder Structure

The repository is organized as a modular monorepo, separating each AI microservice, infrastructure configuration, and supporting documentation. This structure keeps disease detection, yield prediction, and analytics code independently testable and independently containerizable, while sharing a common API gateway layer.

Repository Folder Structure

```
crop-ai-platform/
├── .github/workflows/        # CI pipelines
├── app/
│   ├── disease_detection/    # CNN inference service
│   ├── yield_prediction/     # ML regression service
│   ├── weather_analytics/    # weather impact module
│   ├── api_gateway/          # FastAPI routes
│   └── recommendation/       # treatment engine
├── data/                     # sample datasets (gitignored)
├── models/                   # trained model artifacts
├── notebooks/                # EDA & training notebooks
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── tests/                    # unit & integration tests
├── docs/                     # architecture & API docs
├── .gitignore
├── README.md
└── requirements.txt
```

Figure 2: Repository folder structure

2.2 Purpose of Major Folders and Files

Folder / File	Purpose
app/disease_detection/	CNN-based inference service that classifies leaf images into disease categories.
app/yield_prediction/	ML regression models estimating expected yield from soil, weather, and historical data.
app/weather_analytics/	Processes weather API data to assess disease-risk and yield impact.
app/api_gateway/	FastAPI routing layer that exposes unified REST endpoints to clients.
app/recommendation/	Maps detected disease classes to recommended treatments and dosages.
data/	Sample and reference datasets used for local development (excluded from version control).
models/	Serialized trained model artifacts (e.g., .h5, .pkl) used at inference time.
docker/Dockerfile	Build instructions for containerizing each service.
docker/docker-compose.yml	Multi-container orchestration definition for local and staging deployment.
tests/	Unit and integration tests for each microservice.
docs/	Architecture diagrams, API specifications, and onboarding documentation.

Folder / File	Purpose
.gitignore	Excludes datasets, model binaries, virtual environments, and secrets from Git tracking.
README.md	Project overview, setup instructions, and usage guide.
requirements.txt	Pinned Python dependencies for reproducible environments.

3. Git Repository Initialization

3.1 Initialization Steps

The project repository was initialized locally and linked to a remote host (GitHub) to enable collaborative development among the AI, backend, and DevOps contributors.

```
$ mkdir crop-ai-platform && cd crop-ai-platform
$ git init
$ git branch -M main
$ echo "# Intelligent Crop AI Platform" > README.md
$ git add README.md .gitignore
$ git commit -m "chore: initial commit with README and gitignore"
$ git remote add origin https://github.com/org/crop-ai-platform.git
$ git push -u origin main
```

3.2 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory storing the full object database, refs, and history metadata.
.gitignore	Prevents datasets, trained model binaries, .env secrets, and __pycache__ from being tracked.
README.md	Documents project purpose, setup, and contribution guidelines for new collaborators.
.github/workflows/	Defines CI pipelines that run automatically on push and pull request events.

4. Git Branching Strategy

4.1 Branching Model

The project follows a Gitflow-inspired branching strategy with five branch categories: main, release, develop, feature, and hotfix. This model separates stable production code from active development, allowing multiple team members to build the disease-detection, yield-prediction, and analytics modules in parallel without destabilizing the main branch.

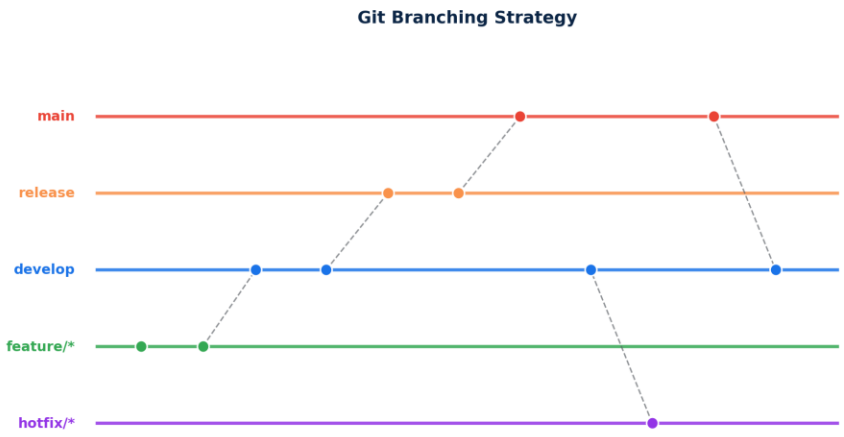

Figure 3: Git branching strategy used in the project

4.2 Branch Roles

Branch	Role
main	Always reflects the production-ready, deployable state of the platform.
release/*	Staging branch for final testing and version tagging before merging to main.
develop	Integration branch where completed features are merged and tested together.
feature/*	Isolated branches for individual features, e.g. feature/disease-cnn, feature/yield-model.
hotfix/*	Branches for urgent production fixes, merged directly into main and back into develop.

4.3 Justification

This model was selected because the platform is composed of several independently developed AI microservices. Feature branches let the vision team, the ML team, and the backend team iterate without conflict, while the develop branch provides a stable integration point for end-to-end testing before a release is cut. The hotfix branch ensures that critical issues in a deployed model (for example, a misclassification bug) can be patched in production quickly without waiting for the next release cycle.

5. Version Control Workflow

5.1 End-to-End Workflow

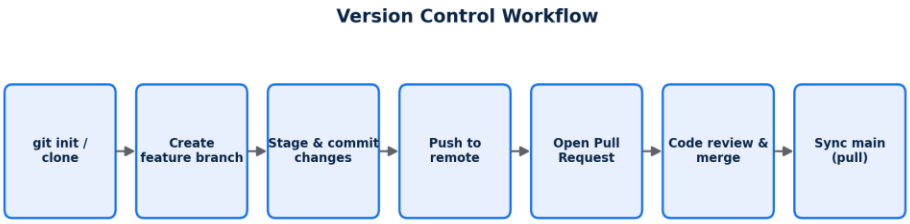

Figure 4: Version control workflow followed for each change

5.2 Demonstration of Git Operations

```
# Create a feature branch for the disease-detection model
$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/disease-detection-cnn

# Stage and commit meaningful changes
$ git add app/disease_detection/model.py
$ git commit -m "feat(disease-detection): add CNN inference pipeline"

# Push branch and open a pull request
$ git push -u origin feature/disease-detection-cnn

# Resolve a merge conflict during integration
$ git checkout develop
$ git merge feature/disease-detection-cnn
# CONFLICT (content): merge conflict in app/api_gateway/routes.py
$ git diff
$ git add app/api_gateway/routes.py
$ git commit -m "merge: resolve routing conflict between CNN and yield modules"

# Synchronize with the remote repository
$ git pull --rebase origin develop
$ git push origin develop
```

5.3 Purpose of Each Operation

- git checkout -b — creates an isolated branch so in-progress AI model work never affects develop or main.
- git add / commit — records a meaningful, reviewable snapshot of the change with an explanatory message.
- git push -u — publishes the branch to the shared remote so teammates and CI pipelines can access it.
- git merge — integrates completed feature work into the shared develop branch.
- Conflict resolution — manual reconciliation of competing edits (e.g., two services touching the same route file) before completing the merge.
- git pull --rebase — keeps local history linear and up to date with the latest changes from teammates before pushing.

6. Commit History Analysis

6.1 Sample Commit History

```
$ git log --oneline --graph --decorate
* 9f3a1c2 (HEAD -> develop) merge: resolve routing conflict between CNN and yield modules
* 7b2e881 feat(disease-detection): add CNN inference pipeline
* 5d19a44 feat(yield-prediction): add regression model for yield estimation
* 3a88bcd feat(weather): integrate weather API for risk scoring
* 1c04ef2 feat(gateway): add unified REST endpoints for all services
* 0a2f9e1 test: add unit tests for disease classification module
* c6d3a90 docs: update README with setup instructions
* 88e12aa chore: initial commit with README and gitignore
```

6.2 Commit Message Conventions

Commits follow the Conventional Commits format: a type prefix (feat, fix, docs, test, chore, refactor) followed by an optional scope and a concise, imperative description. This makes the project history self-documenting and machine-parseable.

Practice	Benefit
Type-prefixed messages (feat, fix, docs...)	Enables automatic changelog generation and quick scanning of history.
Small, atomic commits	Simplifies code review and allows precise reverts without affecting unrelated changes.
Imperative mood ("add", not "added")	Keeps history consistent and readable, matching Git's own commit style.
Scoped messages (e.g. feat(yield-prediction): ...)	Immediately identifies which microservice a change affects.
Referencing issue IDs where applicable	Links commits to tracked requirements for traceability.

Together, these practices make it possible to reconstruct why a change was made, audit the evolution of each AI microservice, and safely roll back a faulty model version if an issue is discovered after deployment.

7. Docker Environment Design

7.1 Why Docker is Suitable for this Project

The platform combines Python-based ML inference services, a REST API gateway, a relational database, and a caching layer, each with different runtime dependencies (TensorFlow/PyTorch, scikit-learn, PostgreSQL drivers). Docker allows each component to be packaged with its exact dependency set, ensuring the disease-detection CNN behaves identically on a developer's laptop, in CI, and in production — eliminating "it works on my machine" issues that are especially common with heavy ML libraries.

7.2 Components Requiring Containerization

- API Gateway — routes external requests to the correct microservice.
- Disease Detection Service — CNN model server with GPU/CPU inference dependencies.
- Yield Prediction Service — scikit-learn / regression model server.

- Weather & Analytics Service — aggregates weather data and generates farm reports.
- PostgreSQL Database — persistent storage for farm, crop, and sensor data.
- Redis Cache — caches frequent predictions and session data for low-latency responses.

8. Dockerfile Development

8.1 Sample Dockerfile — Disease Detection Service

```
FROM python:3.11-slim AS base

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/disease_detection/ ./app/disease_detection/
COPY models/disease_cnn.h5 ./models/disease_cnn.h5

ENV MODEL_PATH=/app/models/disease_cnn.h5
ENV PORT=8001

EXPOSE 8001

HEALTHCHECK --interval=30s --timeout=5s CMD curl -f http://localhost:8001/health || exit 1

CMD ["uvicorn", "app.disease_detection.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

8.2 Purpose of Each Instruction

Instruction	Purpose
FROM python:3.11-slim	Selects a lightweight, reproducible base image with Python pre-installed.
WORKDIR /app	Sets the working directory for all subsequent commands inside the container.
COPY requirements.txt .	Copies dependency manifest first to leverage Docker layer caching.
RUN pip install ...	Installs Python dependencies inside an isolated container layer.
COPY app/... / models/...	Adds application source code and the trained model artifact into the image.
ENV	Defines runtime configuration such as model path and service port.
EXPOSE 8001	Documents the port the service listens on.
HEALTHCHECK	Allows Docker/orchestrators to detect and restart an unresponsive container.
CMD	Specifies the default command that starts the inference server when the container runs.

9. Docker Compose Design

9.1 Multi-Container Architecture

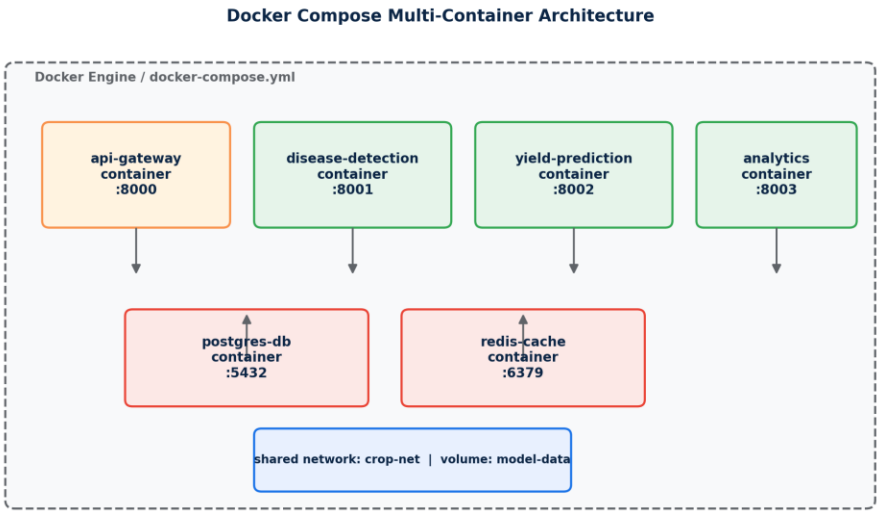

Figure 5: Docker Compose multi-container architecture

9.2 docker-compose.yml

```
version: "3.9"
services:
  api-gateway:
    build: ./docker/api_gateway
    ports: ["8000:8000"]
    depends_on: [disease-detection, yield-prediction, postgres-db]
    networks: [crop-net]

  disease-detection:
    build: ./docker/disease_detection
    ports: ["8001:8001"]
    volumes: ["model-data:/app/models"]
    networks: [crop-net]

  yield-prediction:
    build: ./docker/yield_prediction
    ports: ["8002:8002"]
    volumes: ["model-data:/app/models"]
    networks: [crop-net]

  postgres-db:
    image: postgres:16
    environment:
      POSTGRES_DB: crop_ai
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes: ["pgdata:/var/lib/postgresql/data"]
    networks: [crop-net]

  redis-cache:
```

```
    image: redis:7-alpine
    networks: [crop-net]

networks:
  crop-net:
volumes:
  model-data:
  pgdata:
```

9.3 Services, Networking, Volumes, and Dependencies

- Services: each microservice (api-gateway, disease-detection, yield-prediction) plus postgres-db and redis-cache are defined as independent, individually buildable services.
- Networking: a shared bridge network (crop-net) lets containers resolve each other by service name (e.g. postgres-db) without exposing internal ports publicly.
- Volumes: model-data persists trained model files across container restarts; pgdata persists database contents so data survives container recreation.
- Environment variables: database credentials are injected via a .env file (DB_USER, DB_PASSWORD) rather than hard-coded, keeping secrets out of version control.
- Dependencies: depends_on ensures the API gateway starts only after its upstream services and database are available.

10. Container Deployment and Testing

10.1 Running the Application

```
$ docker compose build
$ docker compose up -d
[+] Running 6/6
 ✓ Network crop-ai-platform_crop-net       Created
 ✓ Container crop-ai-platform-postgres-db   Started
 ✓ Container crop-ai-platform-redis-cache   Started
 ✓ Container crop-ai-platform-disease-det.  Started
 ✓ Container crop-ai-platform-yield-pred.   Started
 ✓ Container crop-ai-platform-api-gateway   Started

$ docker compose ps
NAME                           STATUS          PORTS
crop-ai-platform-api-gateway   Up 12s          0.0.0.0:8000->8000/tcp
crop-ai-platform-disease-det.  Up 12s (healthy) 0.0.0.0:8001->8001/tcp
crop-ai-platform-yield-pred.   Up 12s          0.0.0.0:8002->8002/tcp
crop-ai-platform-postgres-db   Up 13s          0.0.0.0:5432->5432/tcp
crop-ai-platform-redis-cache   Up 13s          0.0.0.0:6379->6379/tcp
```

10.2 Testing Procedure and Evidence

Functional testing was performed by sending sample leaf images and sensor readings to the running containers and verifying end-to-end responses.

```
$ curl -X POST http://localhost:8000/api/diagnose \
    -F "image=@sample_leaf.jpg"
{"disease": "Early Blight", "confidence": 0.94, "recommended_treatment": "Copper-based
fungicide"}

$ curl -X POST http://localhost:8000/api/yield-predict \
    -H "Content-Type: application/json" \
    -d '{"soil_moisture": 32, "temperature": 27, "crop": "wheat"}'
{"predicted_yield_tons_per_ha": 4.6, "confidence_interval": [4.2, 5.0]}

$ docker compose logs api-gateway --tail 5
api-gateway | INFO: 200 OK  POST /api/diagnose      142ms
api-gateway | INFO: 200 OK  POST /api/yield-predict   88ms
```

Both endpoints returned successful (HTTP 200) responses with valid predictions, confirming the containers were correctly networked, the model artifacts loaded successfully via the shared volume, and the API gateway correctly routed requests to each backend microservice.

11. Benefits of Git and Docker

11.1 Benefits Realized in this Project

Area	Benefit for the Crop AI Platform
Collaboration	Multiple contributors developed the CNN model, yield predictor, and gateway in parallel using isolated feature branches.
Version management	Every model and API change is traceable through commit history, enabling audits of when a prediction behavior changed.
Portability	Docker images run identically on a developer laptop, CI runner, or cloud VM regardless of underlying OS or pre-installed libraries.
Deployment	docker compose up reproduces the entire multi-service stack in one command, reducing manual setup errors.
Scalability	Individual services (e.g. disease-detection under high image-upload load) can be scaled independently as container replicas.
Maintainability	Small, well-labeled commits and isolated containers make it easy to locate, fix, and redeploy a faulty component without touching unrelated services.

12. Challenges and Improvements

12.1 Challenges Encountered

- Large trained model files (CNN weights) initially bloated the Git repository, slowing clone times.
- Merge conflicts occurred when the gateway routing file was modified simultaneously by two feature branches.
- Container image size for the disease-detection service was large due to full deep-learning framework dependencies.

- Coordinating environment variables (API keys, DB credentials) securely across multiple containers required care.
- Ensuring consistent model versions between the disease-detection and yield-prediction services during integration testing.

12.2 Suggested Improvements

- Adopt Git LFS (Large File Storage) to manage trained model binaries outside the core repository history.
- Introduce a CODEOWNERS file and mandatory pull-request reviews for shared files like the gateway router.
- Use multi-stage Docker builds and slimmer base images to reduce final image size for ML services.
- Adopt a centralized secrets manager (e.g. Docker secrets or a vault service) instead of plain .env files for production.
- Add automated CI pipelines that run tests and build Docker images on every pull request before merging.
- Introduce model versioning (e.g. via MLflow) so each container always pulls a known, tagged model version.

12.3 Conclusion

Applying Git version control and Docker containerization to the Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System established a reproducible, collaborative, and scalable development workflow. The branching strategy enabled parallel development of independent AI microservices, while Docker Compose allowed the entire multi-service platform to be built, deployed, and tested consistently across environments — laying a solid technical foundation for future enhancements such as automated CI/CD and model versioning.